

MSP Science Demonstration - DIY Instructions

Banana Piano

Materialen:

- Arduino Uno
- Arduino IDE Software
- 8 Bananen (ander fruit werkt ook)
- 8 Weerstanden (2 MΩ)
- Koperdraad
- Speaker
- Soldeerbout
- Doosje voor de electronica

Instructies:

1. Verbind twee draden aan de speaker, een aan de positieve en een aan de negatieve connector.
2. Steek de positieve draad in pin A4 op de Arduino en de negatieve in de aarde (GND).
3. Verbind twee draden aan hetzelfde uiteinde van elke weerstand, een lange om naar de banana te gaan en een korte naar de Arduino.
4. Steek deze laatste draden in pin 3-10 op de Arduino.
5. Verbind de andere uiteindes van de weerstanden samen met korte stukjes draad of soldeer ze aan elkaar vast om 1 uiteinde te krijgen en verbind een langer stuk draad aan het einde hiervan, dit is de "common send" pin.
6. Steek deze pin in pin 2 op de Arduino.
7. Steek de lange draden in de bananen.
8. Installeer de library CapacitiveSensor op Arduino IDE.
9. Kopieer de code uit dit bestand, sla de twee bestanden op in een map en upload de sketch op de Arduino.
10. Raak de bananen aan om de corresponderende frequentie (oftewel de noot) te spelen.



MSP Science Demonstration - DIY Instructions

Banana Piano

Materials:

- Arduino Uno
- Arduino IDE Software
- 8 Bananas (other fruit or materials work too)
- 8 Resistors (2 MΩ)
- Insulated copper wire
- Speaker
- Soldering equipment
- Box for housing the electronics

Instructions:

1. Connect two wires to the speaker, one to the positive and one to the negative connector.
2. Plug the positive wire into pin A4 on the Arduino board and the negative wire into a ground pin (GND).
3. Connect two wires to one end of every resistor, one will go to the banana and the other one to the Arduino.
4. Plug these last wires into pins 3-10 on the Arduino board.
5. String the other ends of the resistors together with short pieces of wire or solder them together to create one strand and connect a longer wire to the end of this string, this is the "common send" pin.
6. Plug this wire into pin 2 on the Arduino board.
7. Stick the wires that go to the banana into the bananas.
8. Install the library CapacitiveSensor on Arduino IDE.
9. Copy the code from this sheet into two files, save them in a folder and upload the sketch onto the Arduino.
10. Touch the banana's to play the corresponding frequency (i.e. the corresponding note).



MSP Science Demonstration - DIY Instructions

Slia deze code op als **CapacitiveSensePiano.ino**
Save this code as **CapacitiveSensePiano.ino**

```
/*
Capacitive-Touch Arduino Keyboard Piano
Plays piano tones through a buzzer when the user taps touch-sensitive piano "keys"
Created 18 May 2013
Modified 23 May 2013
by Tyler Crumpton and Nicholas Jones
This code is released to the public domain. For information about the circuit,
visit the Instructable tutorial at http://www.instructables.com/id/Capacitive-Touch-Arduino-Keyboard-Piano/
*/
#include <CapacitiveSensor.h>
#include "pitches.h"
#define COMMON_PIN 2 // The common 'send' pin for all keys
#define BUZZER_PIN A4 // The output pin for the piezo buzzer
#define NUM_OF_SAMPLES 10 // Higher number whens more delay but more consistent readings
#define CAP_THRESHOLD 150 // Capactive reading that triggers a note (adjust to fit your needs)
#define NUM_OF_KEYS 8 // Number of keys that are on the keyboard
// This macro creates a capacitance "key" sensor object for each key on the piano keyboard:
#define CS(Y) CapacitiveSensor(2, Y)
// Each key corresponds to a note, which are defined here. Uncomment the scale that you want to use:
int notes[]={NOTE_C4,NOTE_D4,NOTE_E4,NOTE_F4,NOTE_G4,NOTE_A4,NOTE_B4,NOTE_C5}; // C-Major scale
//int notes[]={NOTE_A4,NOTE_B4,NOTE_C5,NOTE_D5,NOTE_E5,NOTE_F5,NOTE_G5,NOTE_A5}; // A-Minor scale
//int notes[]={NOTE_C4,NOTE_DS4,NOTE_F4,NOTE_FS4,NOTE_G4,NOTE_AS4,NOTE_C5,NOTE_DS5}; // CBlues scale
// Defines the pins that the keys are connected to:
CapacitiveSensor keys[] = {CS(3), CS(4), CS(5), CS(6), CS(7), CS(8), CS(9), CS(10)};
void setup() {
// Turn off autocalibrate on all channels:
for(int i=0; i<8; ++i) {
keys[i].set_CS_Autocal_Millis(0xFFFFFFFF);
}
// Set the buzzer as an output:
pinMode(BUZZER_PIN, OUTPUT);
}
void loop() {
unsigned int keysPlaying = 0;
// Loop through each key:
for (int i = 0; i < NUM_OF_KEYS; ++i) {
// If the capacitance reading is greater than the threshold, play a note:
int c = keys[i].capacitiveSensor(NUM_OF_SAMPLES);
if(c > CAP_THRESHOLD) {
tone(BUZZER_PIN, notes[i]); // Plays the note corresponding to the key pressed
keysPlaying++;
}
}
if (keysPlaying == 0) {
noTone(BUZZER_PIN);
}
}
```



Maastricht University

Faculty of Science and Engineering

MSP Science Demonstration - DIY Instructions

Slu deze code op als **pitches.h**
Save this code as **pitches.h**

```
// This file contains a list of 8 octaves worth of pitches
// that correspond to notes on the piano.
#define NOTE_B0 31
#define NOTE_C1 33
#define NOTE_CS1 35
#define NOTE_D1 37
#define NOTE_DS1 39
#define NOTE_E1 41
#define NOTE_F1 44
#define NOTE_FS1 46
#define NOTE_G1 49
#define NOTE_GS1 52
#define NOTE_A1 55
#define NOTE_AS1 58
#define NOTE_B1 62
#define NOTE_C2 65
#define NOTE_CS2 69
#define NOTE_D2 73
#define NOTE_DS2 78
#define NOTE_E2 82
#define NOTE_F2 87
#define NOTE_FS2 93
#define NOTE_G2 98
#define NOTE_GS2 104
#define NOTE_A2 110
#define NOTE_AS2 117
#define NOTE_B2 123
#define NOTE_C3 131
#define NOTE_CS3 139
#define NOTE_D3 147
#define NOTE_DS3 156
#define NOTE_E3 165
#define NOTE_F3 175
#define NOTE_FS3 185
#define NOTE_G3 196
#define NOTE_GS3 208
#define NOTE_A3 220
#define NOTE_AS3 233
#define NOTE_B3 247
#define NOTE_C4 262
#define NOTE_CS4 277
#define NOTE_D4 294
#define NOTE_DS4 311
#define NOTE_E4 330
#define NOTE_F4 349
#define NOTE_FS4 370
#define NOTE_G4 392
#define NOTE_GS4 415
#define NOTE_A4 440
#define NOTE_AS4 466
#define NOTE_B4 494
#define NOTE_C5 523
#define NOTE_CS5 554
#define NOTE_D5 587
#define NOTE_DS5 622
#define NOTE_E5 659
#define NOTE_F5 698
#define NOTE_FS5 740
#define NOTE_G5 784
#define NOTE_GS5 831
#define NOTE_A5 880
#define NOTE_AS5 932
#define NOTE_B5 988
#define NOTE_C6 1047
#define NOTE_CS6 1109
#define NOTE_D6 1175
#define NOTE_DS6 1245
#define NOTE_E6 1319
#define NOTE_F6 1397
#define NOTE_FS6 1480
#define NOTE_G6 1568
#define NOTE_GS6 1661
```



Maastricht University

Faculty of Science and Engineering

MSP Science Demonstration - DIY Instructions

```
#define NOTE_A6 1760
#define NOTE_AS6 1865
#define NOTE_B6 1976
#define NOTE_C7 2093
#define NOTE_CS7 2217
#define NOTE_D7 2349
#define NOTE_DS7 2489
#define NOTE_E7 2637
#define NOTE_F7 2794
#define NOTE_FS7 2960
#define NOTE_G7 3136
#define NOTE_GS7 3322
#define NOTE_A7 3520
#define NOTE_AS7 3729
#define NOTE_B7 3951
#define NOTE_C8 4186
#define NOTE_CS8 4435
#define NOTE_D8 4699
#define NOTE_DS8 4978
```

